Exercice 1 : Thème 2 - bases de données –

Un sondage est réalisé sur les élèves entrant en classe de sixième. Il leur est demandé de choisir la langue qu'ils aimeraient apprendre en priorité, sans contrainte de choix.

On répertorie des pays dans lesquels les langues sont parlées afin de constituer la base de données dont le schéma est le suivant :

- **PAYS**(nom_p, capitale, nb_hab, superficie)
- **PARLER**(#nom_p **String**, #id_L **Int**)
- **LANGUE**(id_L **Int**, nom_L **String**, nb_locuteur **Int**, alpha **String**)
- **ELEVE**(id_e **Int**, nom_e **String**, prenom_e **String**, sexe_e **String**, #id_L **Int**)

La base de données est fournie en **Annexe 1**.

On précise que :

- plusieurs langues peuvent être parlées dans un même pays
- plusieurs pays peuvent utiliser la même langue

Partie I : Le modèle relationnel

1. Donner le type de chacun des attributs de la relation *Pays*.
À quel type de contrainte d'intégrité cela fait-il référence ?

2. Les tables PAYS, LANGUE et PARLER étant créées, voici les instructions permettant de créer la table ELEVE :

```
CREATE TABLE ELEVE(id_e int PRIMARY KEY, nom_e varchar(20), prenom_e varchar(20),
sexe_e char(1),
id_L int,FOREIGN KEY (id_L) REFERENCES LANGUE (id_L));
```

a. Que représente l'attribut `id_L` pour la relation LANGUE ?
À quel type de contrainte d'intégrité cela fait-il référence ? En donner la signification dans le contexte de la base de données étudiée.

b. Que représente l'attribut `id_L` pour la relation ELEVE ?
À quel type de contrainte d'intégrité cela fait-il référence ? En donner la signification dans le contexte de la base de données étudiée.

3. Est-il possible d'insérer la ligne `(7, 'Grec', 15, 'Latin')` dans la table LANGUE ? Justifier.

4. **a.** Est-il possible de supprimer le russe de la table LANGUE ? Justifier.
b. Est-il possible de supprimer l'italien de la table LANGUE ? Justifier.

Partie II : Requêtes d'interrogation

5. A partir de tables fournies dans l'annexe 1, donner le résultat des requêtes SQL suivantes :

a.
```
SELECT nom_l
FROM LANGUE
WHERE alpha='Latin';
```

b.
```
SELECT COUNT(*)
FROM PAYS
WHERE capitale LIKE 'B%';
```

6. Donner le sens exact, **en langage naturel**, des requêtes SQL suivantes.
On ne demande pas le résultat de la requête.

a.
```
SELECT nom_p
FROM PAYS
ORDER BY nb_hab;
```

b.
```
SELECT nom_p, nom_l
FROM PARLER
JOIN LANGUE ON PARLER.id_L=LANGUE.id_L
ORDER BY nom_p;
```

c.
```
SELECT nb_hab*100/superficie
FROM PAYS
WHERE nom_p='France';
```

7. Écrire en langage SQL les requêtes qui permettent d'obtenir les affichages suivants :

a. le nom des langues présentes dans la table LANGUE.
b. le nom des langues choisies par au moins un élève.
c. le nom et le prénom des élèves qui veulent apprendre l'Espagnol ou le Catalan (sans distinction de casse).
d. le nom des pays de la table PAYS et leur densité (qui sera nommée "densité") en nombre d'habitants au km^2, classés par densité décroissante.
e. le nombre de pays dont la population est comprise entre 10 et 60 millions d'habitants.

<u>Partie III :</u> Requêtes de modification

8. Donner le sens exact, **en langage naturel**, des requêtes SQL suivantes.
On ne demande pas le résultat de la requête.

a.
```
DELETE FROM ELEVE
WHERE id_L=7;
```

b.
```
UPDATE ELEVE
SET sexe_e='M'
WHERE sexe_e='G';
```

9. Écrire en langage SQL les requêtes qui permettent les mises à jour suivantes :

a. Une erreur a été commise sur la superficie de la Suisse, elle vaut 41 285 km^2.
b. Le flamand, qui s'écrit en alphabet de type latin, est parlé par 2,2 millions de personnes à travers la France, la Belgique et les Pays-Bas. Cette langue a été choisie par les 2 élèves suivants :

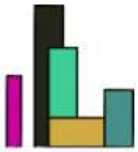
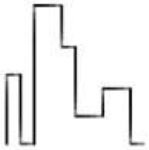
Identifiant (*id_e*)	Nom (*nom_e*)	Prénom (*prenom_e*)	Sexe (*sexe_e*)	Langue choisie (*id_L*)
6	Maillard	Arthur	M	8
7	Guénal	Julie	F	8

La France, dont la capitale est Paris, compte actuellement 67,8 millions d'habitants et sa superficie est de 632 734 km^2.

<u>Attention</u> à l'ordre dans lequel il faut remplir les relations et ajouter ces informations dans la base.

Exercice 2 : thème 4 – langages et programmation, récursivité – thème 5 – algorithmique, méthode « diviser pour régner » –

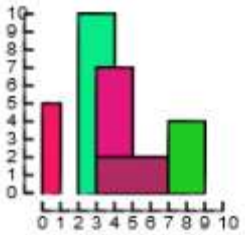
On souhaite dessiner la skyline – contour apparent - d'une ville constituée de plusieurs immeubles.

Les immeubles	La skyline
	

On modélise un immeuble par un triplet (x, y, h) :

- x est l'abscisse du côté gauche de l'immeuble (dans toute la suite $0 \leq x \leq 10$)
- **y est l'abscisse du côté droit**,
- h est sa hauteur.

Une ville est donc une liste d'immeubles.



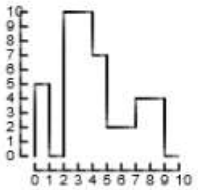
Exemple :

```
ville=[(0,1,5),(2,4,1),(3,5,7),(3,8,2),(7,9,4)]
```

On modélise une skyline par une liste de couples ordonnés par ordre croissant correspondants aux coordonnées des sommets de la ligne.

Exemple :

```
skyline=[(0,0),(0,5),(1,5),(1,0),(2,0),(2,10),(4,10),(4,7),(5,7),(5,2),(7,2),(7,4),(9,4),(9,0),(10,0)]
```



Dans cet exercice, chaque fonction peut-être librement appelée dans une autre fonction même si le code n'est pas donné ou qu'il a été demandé à une question que vous n'avez pas traitée.

Partie I : Dessiner les immeubles et tracer une skyline avec Turtle (Annexe 3 : aide-mémoire Turtle)

1. Quelle instruction Python permet l'utilisation par le programme principal de fonctionnalités issues du module `turtle` ?

2. **a.** La variable ville est définie ainsi :

```
ville=[(0,1,5),(2,4,1),(3,5,7),(3,8,2),(7,9,4)]
```

Quelle est la valeur de `ville[1][2]` ?

Quelle instruction Python permet de récupérer l'abscisse du côté droit du dernier immeuble de la liste ville ?

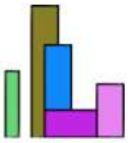
b. Écrire une fonction Python **`trace_immeuble`** à deux paramètres de type tuple (des triplets) : le premier `immeuble` et le second `rgb` qui permet de dessiner de dessiner un immeuble de la couleur `rgb`.

Exemple :

```
trace_immeuble((0,10,50),(255,0,0))
```



c. Écrire une fonction Python **`trace_ville`** à un paramètre de type liste de tuples (liste de triplets) qui dessine les immeubles d'une ville de telle sorte que les immeubles n'aient pas tous la même couleur.



Exemple :

```
trace_ville([(0,1,5),(2,4,10),(3,5,7),(3,8,2),(7,9,4)])
```

3. **a.** La variable **skyline** est définie ainsi :

```
skyline=[(0,0),(0,5),(1,5),(1,0),(2,0),(2,10),(4,10),(4,7),(5,7),(5,2),(7,2)
,(7,4),(9,4),(9,0),(10,0)]
```

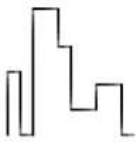
Quelle est la valeur de `skyline[6][1]` ?

Quelle instruction Python permet de récupérer l'ordonnée du dernier sommet de la liste skyline ?

b. Écrire une fonction Python **`trace_skyline`** à un paramètre de type liste de tuples (liste de couples) qui dessine la skyline d'une ville.

Exemple :

```
trace_skyline([(0,0),(0,5),(1,5),(1,0),(2,0),(2,10),(4,10),(4,7),(5,7),(5,2)
,(7,2),(7,4),(9,4),(9,0)]
```



Partie II : Quelques fonctions sur une liste d'immeubles et étude de leur coût

4. Ecriture de fonctions

a. Ecrire une fonction non récursive `hauteur_max_immeubles` à un paramètre de type liste de tuples (liste de triplets) qui renvoie la hauteur maximale d'une liste d'immeubles et `None` si la liste est vide.

Exemple :

```
hauteur_max_immeubles([(0,1,5),(2,4,10),(3,5,7),(3,8,2),(7,9,4)])=10
```

b. Estimer le coût de la fonction `hauteur_max_immeubles` en fonction du nombre d'immeubles.

c. Écrire une fonction récursive `hauteur_max_immeubles_rec` à deux paramètres le premier de type liste de tuples (liste de triplets) et le second de type entier *n* qui renvoie la hauteur maximale de la liste considérée jusqu'à l'indice *n* et None si la liste est vide.

Exemple :

```
ville=[(0,1,5),(2,4,10),(3,5,7),(3,8,2),(7,9,4)]
hauteur_max_immeubles_rec(ville,len(ville))=10
```

5. On admet que l'on a une fonction `calcul_bords` à un paramètre de type liste de tuples (liste de triplets) qui renvoie la liste de tous les bords dans l'ordre croissant et sans redondance.
On ne demande pas d'écrire cette fonction, on admet que son coût est en $nlog(n)$ avec n le nombre d'immeubles de la liste.

Exemple :

```
calcul_bords([(0,1,5),(2,4,10),(3,5,7),(3,8,2),(7,9,4)])=[0, 1, 2, 3, 4, 5, 7, 8, 9]
```

a. Compléter la fonction `dictionnaire_bords_immeubles` **dans l'annexe 2** à un paramètre de type liste de tuples (liste de triplets) qui renvoie un dictionnaire associant à chaque abscisse la liste des numéros d'immeubles ayant un bord à cette abscisse.

Exemple :

```
dictionnaire_bords_immeubles([(0,1,5),(2,4,10),(3,5,7),(3,8,2),(7,9,4)])=
{0: [0], 1: [0], 2: [1], 3: [2,3], 4: [1], 5: [2], 7: [4], 8: [3], 9: [4]}
```

Interprétation :

```
dico={0: [0], 1: [0], 2: [1], 3: [2, 3], 4: [1], 5: [2], 7: [4], 8: [3], 9: [4]}
```

`dico[2]` vaut `[1]` signifie que seul l'immeuble numéro 1 a un bord à l'abscisse 2.

`dico[3]` vaut `[2,3]` signifie que les immeubles numéro 2 et 3 ont un bord à l'abscisse 3.

b. Estimer le coût de la fonction `dictionnaire_bords_immeubles` en fonction du nombre d'immeubles.

<u>Partie III :</u> Calculer une skyline en utilisant la méthode « diviser pour régner »

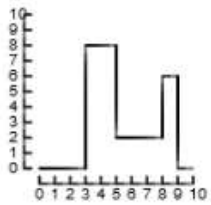
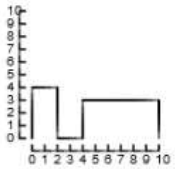
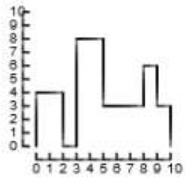
Il s'agit d'une partie d'algorithmique. Les algorithmes sont proposés en langage naturel.

Pour représenter une liste d'immeubles et une skyline on utilise les notations Python :

- liste d'immeubles : `[(0,1,5),(2,4,10),(3,5,7),(3,8,2),(7,9,4)]`
- skyline :
  `[(0,0),(0,4),(2,4),(2,0),(3,0),(3,8),(5,8),(5,3),(8,3),(8,6),(9,6),(9,3),(10,3),(10,0)]`

On admet que l'on a une fonction **fusion** à deux entrées des skylines qui renvoie la skyline obtenue à partir des deux autres. **On ne demande pas d'écrire cette fonction**.

<u>Exemple :</u>

Skylines d'entrée	`[(0,0),(0,4),(2,4),(2,0),(4,0),(4,3),(10,3),(10,0)]`	
	`[(0,0),(3,0),(3,8),(5,8),(5,2),(8,2),(8,6),(9,6),(9,0),(10,0)]`	
Skyline de sortie	`[(0,0),(0,4),(2,4),(2,0),(3,0),(3,8),(5,8),(5,3),(8,3),(8,6),(9,6),(9,3),(10,3),(10,0)]`	

Ci-dessous on donne en langage naturel la fonction **calcul_skyline**. Cette fonction a une entrée une liste d'immeubles d'une ville et renvoie la skyline correspondante.

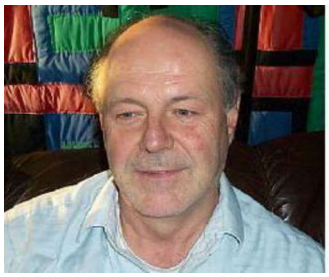
```
calcul_skyline(ville):
    si ville est vide :
      retourner [(0,0),(10,0)]
    sinon :
      si ville a un batiment(x,y,h) :
        retourner [(0,0),(x,0),(x,h),(y,h),(y,0),(0,10)]
      sinon:
        ville1 ← sous-liste composée de la moitié[1] des éléments de ville
        ville2 ← sous-liste composée de l'autre moitié des éléments de ville
        retourner fusion(calcul_skyline(ville1),calcul_skyline(ville2))
```

[1] À un élément près

6. L'algorithme proposé pour calculer la skyline utilise le principe « diviser pour régner ».
   a. Expliquer la méthode « diviser pour régner » dans le cas général.
   b. Citer deux exemples d'algorithmes utilisant ce principe.
   c. Justifier que la fonction `calcul_skyline` proposée page 7 est basée sur ce principe.

7. Représenter l'évaluation de l'appel à `skykine([(0,1,5),(2,4,10)])` par un arbre d'appel faisant apparaître les appels aux fonctions `calcul_skyline` et `fusion` (numérotés en bleu) et aussi les retours de ces appels (numérotés d'une autre couleur).

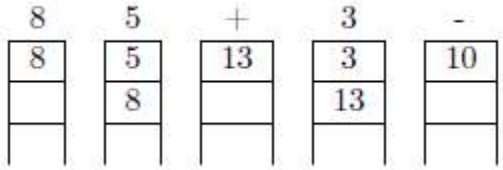
Exercice 3 : thème 1 – structures de données, vocabulaire POO et structures linéaires -



Le langage Forth a été créé au début des années 1960 par Chuck Moore (né en 1938, photo ci-jointe). C'est un langage important du point de vue de l'histoire de l'informatique, d'une part parce qu'il a été développé pour l'informatique embarquée (suivi de satellites, guidage de télescopes...), d'autre part parce qu'il est basé sur la structure de pile (stack).

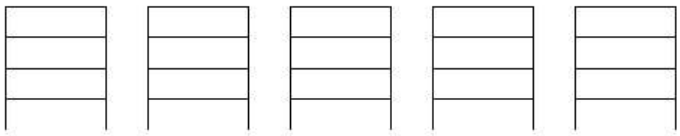
I. Notation polonaise inversée

La pile de Forth est utilisée pour évaluer des expressions algébriques comme : **8 5 + 3 –** écrites en notation polonaise inversée (RPN[2]).
L'expression **8 5 + 3 –** s'évalue à 10 comme le montrent les états successifs de la pile de Forth :



On empile la valeur 8.
On empile la valeur 5.
On remplace les deux nombres du sommet de la pile (8 et 5) par leur somme 13.
On empile la valeur 3.
On remplace les deux nombres du sommet de la pile (13 et 3) par leur différence 10.

1. Que donne l'évaluation par Forth, de l'expression **2 3 – 5 +** ?
   On pourra s'aider de ces dessins d'états successifs de la pile de Forth que l'on recopiera sur la copie :



Une conséquence importante de l'utilisation d'une pile est la quasi-suppression des noms de variables locales, qui deviennent inutiles.

II. Les instructions Forth à partir de l'interface d'une pile

L'interpréteur de Forth est basé sur une pile (stack) dont l'interface est donnée ci-dessous :

Interface d'une pile d'éléments de type `T` , notée `Stack[T]` -

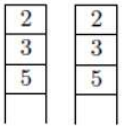
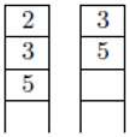
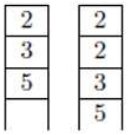
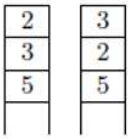
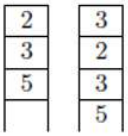
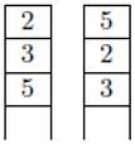
Fonction	Description
`create_stack() -> Stack[T]`	crée une pile vide
`empty(s : Stack[T]) -> bool`	renvoie True si la pile s est vide et False sinon
`push(s : Stack[T], e :T) -> None`	ajoute l'élément e au sommet de la pile s
`pop(s : Stack[T]) -> T`	retire et renvoie l'élément au sommet de la pile s

[2] Reversed Polish Notation : proposée par Arthur Burks (1915-2008) en 1945, sur l'ordinateur ENIAC.

2. **a.** Le mode LIFO permet de décrire la structure d'une pile. Que signifie cet acronyme ?
b. Que signifie l'acronyme FIFO ?
A quelle structure de données associe-t-on ce mode ?

Outre empty, push et pop la pile de Forth possède les instructions décrites dans le tableau ci-dessous. L'interface des différentes fonctions est donnée dans les deux premières colonnes et la colonne exemple donne, à partir du même état initial de la pile s 

, et son état après l'exécution de l'instruction.

Fonction	Description	Exemple
`top(s : Stack[T]) -> T`	renvoie l'élément au sommet de la pile s sans la modifier.	
`drop(s : Stack[T]) -> None`	supprime l'élément au sommet de la pile s.	
`dup(s : Stack[T]) -> None`	duplique le haut de la pile s.	
`swap(s : Stack[T]) -> None`	échange les contenus des deux cellules du haut de la pile s.	
`over(s : Stack[T]) -> None`	duplique l'avant-dernier élément de la pile, en plaçant sa copie au-dessus de la pile s.	
`rot(s : Stack[T]) -> None`	déplace l'avant-avant-dernier élément de la pile s vers le haut de celle-ci (ce qui revient à une rotation de ses trois derniers éléments).	

3. En utilisant uniquement les fonctions `empty`, `push` et `pop` on peut écrire toutes les autres fonctions.

Exemples :

```
def top(s):
    x = pop(s)
    push(s,x)
    return x
```

```
def drop(s):
    x = pop(s)
    return None
```

```
def dup(s):
    x = pop(s)
    push(s,x)
    push(s,x)
    return None
```

Écrire les fonctions `swap`, `over` et `rot` en utilisant uniquement les fonctions `empty`, `push` et `pop`.

4. On considère la séquence d'instructions donnée ci-dessous :

```
swap(s)
dup(s)
rot(s)
swap(s)
```



   a. Décrire les états successifs de la pile lors de l'exécutions de cette séquence.
   b. Donner une description de l'instruction qui permet de passer de l'état initial à l'état final.

III. Implémentation d'une pile

On propose une implémentation en Python de cette pile s à partir d'une liste chainée;
Chaque élément de la liste chainée est matérialisé par un objet de la classe Cellule.

```
class Cellule:
    """une cellule d'une liste chaînée"""
    def __init__(self,v,s):
        self.valeur=v
        self.suivante=s

class Stack:
    """ structure de Pile implémentée avec les listes chainées"""
    def __init__(self):
        self.contenu=None

    def empty(self):
        return self.contenu==None

    def push(self,element):
        self.contenu=Cellule(element,self.contenu)

    def pop(self):
        assert(not self.empty())
        element=self.contenu.valeur
        self.contenu=self.contenu.suivante
        return element
```

5. Préciser les attributs de chacune des classes `Cellule` et `Stack.`
6. Quel nom donne-t-on à la méthode spéciale `__init__` que l'on trouve dans chacune des classes ?
7. Quelles sont les méthodes de la classe `Stack` ?
8. Indiquer les instructions Python qui permettent créer une pile nommée p, composée des éléments suivants : 15, 12, 9, 35 et 28 (le sommet de la pile est 28).
9. **a.** Dans la méthode pop on peut lire `assert(not self.empty())`. Expliquer cette ligne.
   b. Proposer une solution qui permette d'alerter l'utilisateur sans interrompre l'exécution du programme.

10. a. Qu'est-ce qu'une interface ?
Qu'est-ce qu'une implémentation ?

b. Écrire une classe `Stack2` satisfaisant l'interface (rappelée ci-dessous) d'une pile d'éléments qui utilise un tableau Python avec les opérations append et pop.

Interface d'une pile d'éléments de type `T` , notée `Stack[T]` -

Fonction	**Description**
`create_stack() -> Stack[T]`	crée une pile vide
`empty(s : Stack[T]) -> bool`	renvoie True si la pile s est vide et False sinon
`push(s : Stack[T], e :T) -> None`	ajoute l'élément e au sommet de la pile s
`pop(s : Stack[T]) -> T`	retire et renvoie l'élément au sommet de la pile s

Annexe 1

PAYS			
Nom (*nom_p*)	Capitale (*capitale*)	Nombre d'habitants en millions (*nb_hab*)	Superficie en km² (*superficie*)
Allemagne	Berlin	83,1	357 386
Argentine	Buenos Aires	44,7	2 791 810
Belgique	Bruxelles	11,4	30 688
Espagne	Madrid	46,9	505 911
Grèce	Athènes	10,7	131 957
Italie	Rome	60,5	301 336
Pays-Bas	Amsterdam	17,1	41 530
Suisse	Berne	8,5	412 850

LANGUE			
Identifiant (*id_L*)	Nom (*nom_L*)	Nombre de locuteurs en millions (*nb_locuteur*)	Alphabet (*alpha*)
1	Espagnol	460	Latin
2	Catalan	10	Latin
3	Néerlandais	23	Latin
4	Allemand	115	Latin
5	Italien	70	Latin
6	Russe	280	Cyrillique
7	Grec	15	Grec

ELEVE				
Identifiant (*id_e*)	Nom (*nom_e*)	Prénom (*prenom_e*)	Sexe (*sexe_e*)	Langue choisie (*id_L*)
1	Grange	Justine	F	4
2	Baillouv	Maxime	M	5
3	Thomain	Lucas	M	1
4	Padovani	Sarah	F	2
5	Pierson	Manon	F	3

PARLER			
nom_p	*id_L*	*nom_p*	*id_L*
Suisse	4	Pays-Bas	3
Suisse	5	Allemagne	4
Espagne	1	Italie	5
Espagne	2	Argentine	1
Belgique	4	Belgique	3

Nom, Prénom : …… **Annexe 2**

A rendre avec la copie

```
def dictionnaire_bords_immeubles(ville):
    """à partir d'une liste de triplets (x,y,h) - x et y sont les bords gauche et droit de l'immeuble et h sa hauteur-
    renvoie un dictionnaire associant à chaque abscisse la liste des numéros d'immeubles ayant un bord sur cette abscisse"""
    # calcul de la liste de toutes les abscisses des bords des immeubles sans redondance et par ordre croissant
    lst=….
    # création d'un dictionnaire vide
    dico=…
    # initialisation de ce dictionnaire : pour clefs les abscisses de tous les bords et pour valeurs une liste vide
    …




    # parcours de la liste d'immeubles et ajout dans les valeurs du dictionnaire du numéro d'immeuble qui a un bord à l'abscisses considérée
    for num_imm in range (len(ville)):
        …




    # renvoie le dictionnaire
        return dico
```

Annexe 3

©2016 - Laurent Pointal
Licence Creative Commons Paternité 4

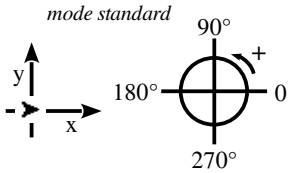
Aide Mémoire V4

Aide Mémoire Turtle

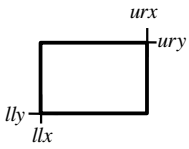
Dernière version sur :
https://perso.limsi.fr/pointal/python:turtle

Coordonnées / environnement

Par **défaut**, en mode *standard*, direction initiale vers la droite, utilisation d'un **repère cartésien orthonormé**, angles **sens trigonométrique** et en **degrés**.
La position 0,0 est placée au centre de la fenêtre.
En mode *logo*, la direction initiale est vers le haut et les angles sont positifs dans le sens des aiguilles d'une montre (sens inverse trigo).
En mode *world*, unités pixels, le repère n'est pas nécessairement normé (pixels non carrés).



`degrees()`	expression des angles en degrés (tour=360°)
`degrees(`*n*`)`	expression des angles unité au choix (tour=n)
`radians()`	expression des angles en radians (tour=2π = 2×3.14…rad)
`mode(`*m*`)`	fixe le mode de coordonnées : `"standard"`, `"logo"`, `"world"`
`title(`*t*`)`	fixe le titre de la fenêtre
`screensize()` → *(larg,haut)*	dimensions de la fenêtre
`screensize(`*l*`,`*h*`[,`*coul*`])`	fixe dimensions de la fenêtre et couleur de fond
`setup(`…`)`	fixe position et dimensions de la fenêtre
`window_width()` → *larg*	largeur de la fenêtre
`window_height()` → *haut*	hauteur de la fenêtre



`setworldcoordinates(`*llx*`,`*lly*`,`*urx*`,`*ury*`)`	fixe système de coordonnées (fait un `reset()`)
`bgcolor(`*[coul]*`)`	fixe/rend couleur du fond
`bgpic(`*[nom]*`)`	fixe/rend l'image de fond (nom fichier gif, `'nopic'` pour supprimer l'image)

Formes

- `"arrow"`
- `"turtle"`
- `"circle"`
- `"square"`
- `"triangle"`
- `"classic"`

Utilisées aussi comme tampons (cf `stamp()`).

python powered

Couleurs

Turtle utilise les noms des couleurs de Tk, dont voici un petit extrait.

- `"black"`
- `"white"`
- `"grey"`
- `"red"`
- `"orange"`
- `"green"`
- `"blue"`
- `"navy"`
- `"yellow"`
- `"gold"`
- `"tan"`
- `"brown"`
- `"sienna"`
- `"wheat"`
- `"cyan"`
- `"pink"`
- `"salmon"`
- `"violet"`
- `"purple"`

Collection des couleurs sur `http://wiki.tcl.tk/37701`
Noms+valeurs RGB sur `https://www.tcl.tk/man/tcl8.6/TkCmd/colors.htm`

Codes RGB

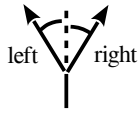
r=rouge g=vert b=bleu
(red) (green) (blue)

Via une chaîne de valeurs hexa, composantes sur 4/8/12 bits :
`"#rgb"`
`"#rrggbb"`
`"#rrrgggbbb"`

Ou via tuple de 3 flottants entre 0.0…1.0 ou de 3 entiers entre 0…255:
`(r,g,b)`

Voir `colormode()`

Position & Déplacements



`forward(`distance`)`	avance `fd`
`backward(`distance`)`	recule `bk` `back`
`left(`angle`)`	tourne à gauche `lt`
`right(`angle`)`	tourne à droite `rt`
`setposition(`x`,`y`)`	vas à la position x,y `setpos` `goto`
`setx(`x`)`	vas à l'abscisse x
`sety(`y`)`	vas à l'ordonnée y
`home()`	vas à l'origine 0,0
`setheading(`angle`)`	s'oriente à l'angle `seth`
`circle(`rayon*[*`,`angle*]*`,`pas*]*`)`	cercle/arc/polygone
`position()` → *(x, y)*	position courante `pos`
`xcor()` → *x*	abscisse courante
`ycor()` → *y*	ordonnée courante
`distance(`*x*`,`*y*`)` → *d*	calcul distance jusqu'à x,y
`distance(`*pos*`)` → *d*	calcul distance jusqu'à pos (x,y)
`heading()` → *a*	orientation courante (angle)
`towards(`*x*`,`*y*`)` → *a*	calcul angle vers x,y
`towards(`*pos*`)` → *a*	calcul angle vers pos (x,y)
`dot(`*[*taille*[*`,`coul*]]*`)`	trace point à la position
`stamp()` → *id*	trace tampon tortue à la position
`clearstamp(`id`)`	efface tampon id
`clearstamps(`*[*n*]*`)`	efface tampons (tous, n>0 premiers n<0 n derniers
`undo()`	annuler dernier mouvement/trace

Pinceau

`penup()`	lève (pas de trace) `up` `pu`
`pendown()`	baisse (trace) `down` `pd`
`isdown()` →*état*	retourne vrai si pinceau baissé
`color(`cp*[*`,`cr*]*`)`	fixe/rend couleur du pinceau [et du remplissage]
`pencolor(`coul`)`	fixe/rend couleur du pinceau
`fillcolor(`coul`)`	fixe/rend couleur du remplissage
`pensize(`larg`)`	largeur du trait `width`
`pen()` →*p*	dico caractéristiques pinceau
`pen(`p`)`	fixe caractéristiques pinceau via dico
`filling()` →*b*	vrai si remplissage actif
`begin_fill()`	démarre tracés de remplissage… *instructions de déplacements*
`end_fill()`	…termine et remplissage des tracés
`colormode()` →*n*	valeur maximale pour les `r g b`
`colormode(n)`	1 ou 255 - val maxi pour les `r g b`

Contrôles

`end()`
libère la fenêtre de la tortue
`bye()`
ferme la fenêtre de la tortue
`reset()`
réinitialisation complète `resetscreen`
`clear()`
effacement de la zone de tracé `clearscreen`
`tracer()` →*n*
périodicité animation tortue
`tracer(`n*[*`,`d*]*`)`
fixe périodicité animation tortue
`delay()` →*n*
délai (ms) entre mises à jour
`delay(`delay`)`
fixe délai (ms) entre mises à jour
`update()`
force mise à jour
`speed()` →*n*
vitesse de tracé
`speed(`*n*`)`
fixe/rend vitesse tracé *n*, nom ou entier [0…10]
`"fastest":0 "fast":10 "normal":6 "slow":3 "slowest":1`
`hideturtle()`
masque la tortue `ht`
`showturtle()`
affiche la tortue `st`
`isvisible()` →*v*
vrai si tortue visible
`shape(`nom`)`
fixe la forme de la tortue
`getshapes()` →*[nom]*
liste des noms de formes
`register_shape(`nomfichier`)`
enregistre forme via fichier gif
`register_shape(`nom`,`coords`)`
enregistre forme via liste de (x,y)
`register_shape(`nom`,`shape`)`
enregistre forme via objet `Shape`
`resizemode()` →*rmode*
mode redimensionnement tortue
`resizemode(`rmode`)`
change le mode `"auto"` `"user"` `"noresize"`